PGBD - PROJET DE DÉVELOPPEMENT SGBD

Architectures applicatives:
Principes de base et bonnes pratiques (partie 1)

Frédéric Parion
f.parion@esa-namur.be

1

ARCHITECTURES APPLICATIVES

Réflexions préliminaires via une étude de cas

2

Architectures applicatives

Etude de cas (1/5)

Mini Workshop:

Etude de cas basée sur la base de données Chinook.

Réflexion et construction progressive d'une application. Ajout de fonctionnalités, refactoring du code et adaptation de l'architecture.

3

3

Architectures applicatives

Etude de cas (2/5)

Fonctionnalités nécessaires :

- Gérer les genres musicaux (CRUD)
- Afficher les informations complètes d'un artiste
- Créer une facture pour l'achat de morceaux musicaux
- Connaitre le niveau d'importance d'un client donné pour l'entreprise Chinook
- Attribuer un vendeur à un client

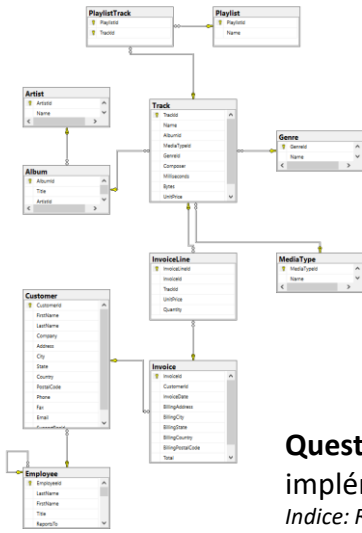
Question: Comment organiseriez-vous le code pour implémenter ces fonctionnalités (app. console) ?

4

4

Architectures applicatives

Etude de cas (3/5)



Fonctionnalités nécessaires :

- Gérer les genres musicaux (CRUD)
- Afficher les informations complètes d'un album
- Créer une facture pour l'achat de morceaux musicaux
- Connaitre le niveau d'importance d'un client donné pour l'entreprise Chinook
- Attribuer un vendeur à un client

Question: Comment organiseriez-vous le code pour implémenter ces fonctionnalités (app. console) ?

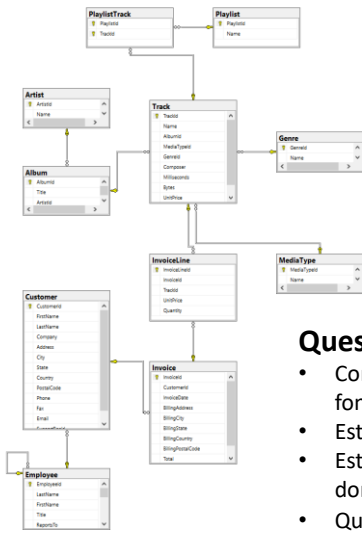
Indice: Repartir du schéma et essayer de regrouper les tables de façon logique.

5

5

Architectures applicatives

Etude de cas (4/5)



Fonctionnalités nécessaires :

- Afficher les informations complètes d'un album
- Gérer les genres musicaux (CRUD)
- Créer une facture pour l'achat de morceaux musicaux
- Connaitre le niveau d'importance d'un client donné pour l'entreprise Chinook
- Attribuer un vendeur à un client

Questions:

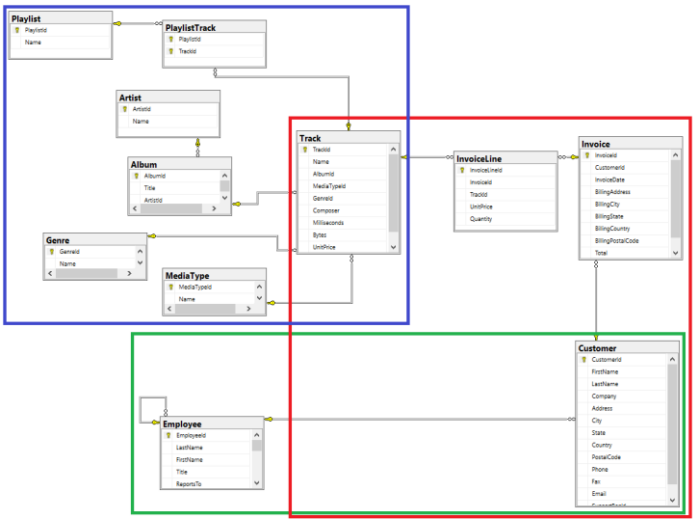
- Comment organiseriez-vous le code pour implémenter ces fonctionnalités (app. console) ?
- Est-ce que tout serait dans le code d'affichage console ?
- Est-ce la responsabilité du code console de gérer les accès à la base de données ?
- Que faire si on a besoin de certaines fonctions à plusieurs endroits ?

6

6

Architectures applicatives

Etude de cas (5/5)



Encapsulation par 3 classes de services

- MusicService
- InvoicingService
- SalesService

7

7

Architectures applicatives

Etude de cas (4/4)

Détails voir code du workshop…

8

8

ARCHITECTURES APPLICATIVES

Exemple d'une classe business de service

9

Architectures applicatives

Exemple de classe business (1/5)

Récupération d'une liste d'entités:

```
public static List<DAL.DB.Artist> GetArtists()
{
    using (DAL.DB.Chinook2Context db = new
      DAL.DB.Chinook2Context())
    {
        return db.Artists.ToList();
    }
}

var artistes = BU.MusicService.GetArtists();
```

10

10

Architectures applicatives

Exemple de classe business (2/5)

Récupération d'une entité:

```
public static DAL.DB.Artist? GetArtist(int artistID)
{
    using (DAL.DB.Chinook2Context db = new
             DAL.DB.Chinook2Context())
    {
        return db.Artists.SingleOrDefault(a => a.ArtistId == artistID);
    }
}

var artiste = BU.MusicService.GetArtist(50);
```

11

11

Architectures applicatives

Exemple de classe business (3/5)

Ajout d'une entité:

```
public static int AddArtist(DAL.DB.Artist newArtist)
{
    using (DAL.DB.Chinook2Context db = new DAL.DB.Chinook2Context())
    {
        db.Artists.Add(newArtist);
        db.SaveChanges();
    }
    return newArtist.ArtistId;
}

BU.MusicService.AddArtist(new DAL.DB.Artist() { Name = "Nirvana" });
```

12

12

Architectures applicatives

Exemple de classe business (4/5)

Update d'une entité:

```
public static void UpdateArtist(DAL.DB.Artist artist)
{
    using (DAL.DB.Chinook2Context db = new DAL.DB.Chinook2Context())
    {
        db.Update(artist);
        db.SaveChanges();
    }
}
var artiste = BU.MusicService.GetArtist(50);
artiste.Name = "Metallica";
BU.MusicService.UpdateArtist(artiste);
```

13

13

Architectures applicatives

Exemple de classe business (5/5)

Suppression d'une entité:

```
public static void RemoveArtist(int artistID)
{
    using (DAL.DB.Chinook2Context db = new DAL.DB.Chinook2Context())
    {
        var artistToRemove = db.Artists.SingleOrDefault(a =>
                    a.ArtistId == artistID);
        if (artistToRemove != null)
        {
            db.Artists.Remove(artistToRemove);
            db.SaveChanges();
        }
    }
}
```

14

14

ARCHITECTURES APPLICATIVES

Introduction aux architectures en couches
(layered architectures)

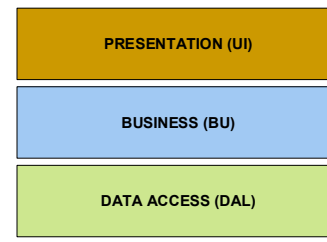
15

Introduction

Définition (1/2)

Les architectures en couches (layered architectures) sont des modèles logiques d'architectures applicatives qui visent à grouper, les fonctionnalités similaires d'une application, en couches logicielles distinctes. L'application est alors présentée comme un empilement de n couches logicielles dont le rôle (responsabilité) est clairement défini. On distingue au minimum trois couches principales:

- La couche **présentation**
- La couche **métier**
- La couche **d'accès aux données** (ou infrastructure)

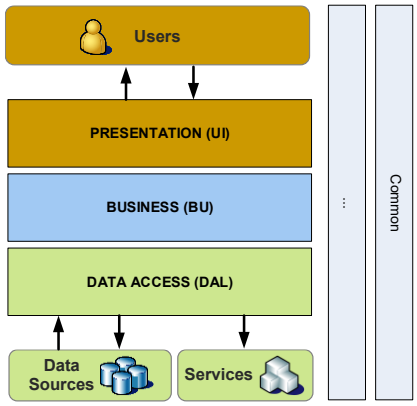


16

16

Introduction

Définition (2/2)



- **UI**: Interface de l'application, gère l'interaction aves les utilisateurs.
- **BU**: Partie fonctionnelle de l'application qui contient la logique applicative et métier.
- **DAL/INFRASTRUCTURE**: Gère l'accès aux sources de données.

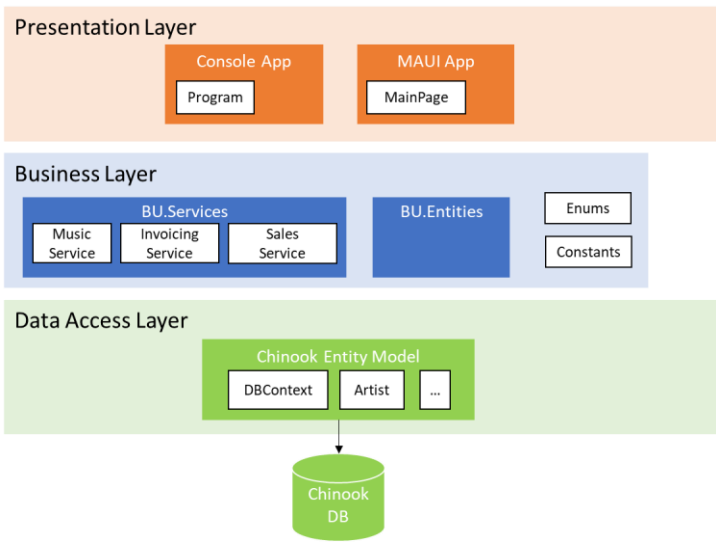
- Une couche offre des *services* à la couche supérieure.
- La communication avec les couches voisines peut être stricte ou plus souple.
- Il peut exister des couches transversales

17

17

Introduction

Architecture du Workshop



18

18

Introduction

Objectifs de ces architectures (1/2)

- Séparation claire entre UI, logique applicative (métier) et accès aux données:
  - Allège le client
  - Favorise la réutilisation (ex: ajout d'un autre type de UI)
  - Facilite la gestion des changements (maintenance):
    - les redondances de code sont évitées-> modifications centralisées
    - modification dans un couche n'a pas d'impact autre couche (sauf…)
    - changement de la stratégie de stockage sans impacter UI et BU
    - …
- Prendre en compte l'hétérogénéité des plateformes: serveurs, clients, langages, …

19

19

Introduction

Objectifs de ces architectures (2/2)

- Amélioration des performances et possibilité d'évolution de la charge. Lorsque les couches sont séparées physiquement on parle d'architecture **multi-tier**.
- Facilitation de la mise en place des politiques de sécurité.
- Des fonctionnalités business peuvent être mises à disposition sans passer par une application client traditionnelle via des **web services**.

Dans le cours de PIDV on étudiera un modèle à 4 couches.

20

20